ARCHITECTURE COMPLIANCE AUDIT

Eight components built, one organism not yet assembled

Comparison of `spawn/` against `Architecture.md` — the Aether-class first-principles design. Read-only pass across all eight components plus cross-cutting architecture guarantees.

Overall completion **53%** Tests **134 / 134** passing Components scaffolded **8 / 8**

Components fully complete **2 / 8** Composition root **not wired**

Completion by component

Percentage weights each named sub-responsibility equally within a component (COMPLETE = 1, PARTIAL = 0.5, MISSING = 0), matching the checklist the audit was scoped against.

COMPONENT	COMPLETE	PARTIAL	MISSING	SCORE
Kernel	1	0	5	17%
Perception	2	1	0	83%
World Model	4	0	0	100%
Executive	0	3	2	30%
Governor	1	3	0	63%

Executor	1	0	3	25%
Memory & Ledger	0	1	7	6%
Inference Port	3	0	0	100%

01 Kernel 17%

RESPONSIBILITY	STATUS	NOTES
event log semantics	COMPLETE	Append-only `EventLog` with monotonic sequence numbers, `read_all`/`read_from`, FIFO dispatch via a queue that respects start/stop lifecycle. 15 kernel tests cover ordering, nested publishes, and shutdown draining.

`scheduler`	MISSING	**Why:** never implemented. **Requirement:** architecture lists "scheduling" as a first-class Kernel responsibility alongside the event loop — nothing currently triggers work on its own; every event is caused by a direct method call from a test or another component. **Existing support:** `run_until_idle` gives a synchronous drain loop a scheduler could sit on top of. **Classification:** extension of Kernel.
`timers`	MISSING	**Why:** the "timer table" Kernel is supposed to own per the architecture table doesn't exist as data or code. **Requirement:** a way to register "fire at/after T" work (e.g. World Model's decay currently must be invoked manually by a caller instead of firing on its own). **Existing support:** none. **Classification:** new feature.

`replay`	MISSING	**Why:** `EventLog.read_from` exists structurally but nothing consumes it to rebuild component state. **Requirement:** "Restart = re-read log tail" from the architecture's crash-recovery story. **Existing support:** the read primitive is there; no component subscribes to its own historical events on startup. **Classification:** extension of Kernel + every store-owning component.
`crash recovery`	MISSING	**Why:** depends on persistence and replay, neither of which exist. **Requirement:** axiom 1, "kill every process, restart from disk, same organism." **Existing support:** none beyond in-memory state. **Classification:** new feature, blocking.
`snapshotting`	MISSING	**Why:** `data/snapshots/` is an empty stub package — zero serialization code anywhere in the repo. **Requirement:** "event log growth needs compaction with snapshotting" (§9). **Existing**

support: directory scaffold only. **Classification:** new feature.

02 Perception 83%

RESPONSIBILITY	STATUS	NOTES
sensor registry	COMPLETE	Register/unregister/get_metadata/is_active, fully tested including duplicate-registration and unknown-sensor errors.
normalization	PARTIAL	**Why:** `Observation.normalized_value` is supplied by the caller as-is — `record_observation` does no unit conversion,

		scaling, or transform. **Requirement:** Perception's stated job is turning *raw* signals into normalized ones; today it's a pass-through. **Existing support:** the field and validation (confidence range check, active-sensor check) are in place; only the transform step is missing. **Classification:** extension of Perception.
`observation ownership`	COMPLETE	Immutable `Observation`, append-only `ObservationLog` with sequence numbers, `ObservationCreated` published through the Kernel on every accepted reading.

03 World Model

100%

RESPONSIBILITY	STATUS	NOTES
`confidence`	COMPLETE	Updates on repeat observation of the same belief via averaging against the new reading. Simpler than the architecture's aspirational "Bayesian-style, source reliability tracked per sensor" fast loop (§7) — flagged under Memory & Ledger's fast-loop row and the roadmap, not deducted here since it satisfies the World Model task's own scope.
`provenance`	COMPLETE	Every belief carries `provenance` = the contributing `sensor_id`, set on creation and preserved across updates.
`decay`	COMPLETE	Linear time-based decay via `apply_decay(now)`, floors at zero, emits `BeliefUpdated` only when confidence actually moves. Matches the "simple time-based decay" the task asked for.

belief ownership	COMPLETE	Immutable `Belief`, current-state `BeliefStore` keyed by belief_id, exclusively owned and mutated by World Model.

04 Executive

30%

RESPONSIBILITY	STATUS	NOTES
opportunity generation	PARTIAL	**Why:** every `BeliefCreated/BeliefUpdated` deterministically spawns exactly one `Opportunity` from that single belief — no aggregation across related beliefs, no goal-

		linkage, no novelty check against episodic memory. **Requirement:** "opportunity discovery is open-ended" (§1); §9 explicitly calls out a "degenerate opportunity generation" failure mode with a named mitigation (novelty penalty) that isn't built. **Existing support:** the reactive pipeline, `OpportunityStore`, and `OpportunityIdentified` event are all in place as the substrate to extend. **Classification:** extension of Executive.
`expected value estimation`	PARTIAL	**Why:** EV is `confidence × 100`, a fixed linear formula with no economic modeling, no goal-value weighting, and no use of the now-complete Inference Port. **Requirement:** §7's calibration loop assumes EV estimates come from judgment (LLM-backed) that can be scored and discounted for

		optimism bias — there's no judgment call to calibrate yet. **Existing support:** `OpportunityScored` event and the scoring step exist; only the estimator itself is a stub. **Classification:** extension of Executive, depends on Inference Port integration.
`planning`	PARTIAL	**Why:** every plan gets the same fixed two-step `ordered_actions` regardless of opportunity content; no ranking or selection among candidates. `PlanAbandonedEvent` and `ExecutiveDecisionEvent` (PLAN_SELECTED) are defined in the event taxonomy but never published anywhere in the codebase — confirming "generate, rank, plan" (§10) is only "generate." **Existing support:** `PlanStore`, `PlanProposed`, and the dead event types are ready to be wired up. **Classification:** extension of Executive.

`attention allocation`	MISSING	**Why:** `attention_cost` is a hardcoded constant (1.0) on every plan; nothing weighs competing opportunities against a shared attention budget. **Requirement:** Executive is supposed to "allocate attention" per the architecture's responsibility table. **Existing support:** `GoalStore` exists but Executive never reads or writes to it — there is no goal tree driving prioritization at all. **Classification:** new feature (goal-tree-driven allocation).
`capital allocation`	MISSING	**Why:** `capital_cost` is a fixed 10% of expected_value; Executive never reads Governor's live `BudgetState` before proposing, so it can (and does, in tests) propose plans the Governor immediately rejects for insufficient funds. **Requirement:** real capital allocation across multiple open

plans, portfolio-aware. **Existing support:** Governor's `BudgetChecked` event already carries the numbers Executive would need. **Classification:** new feature.

05 Governor

63%

RESPONSIBILITY	STATUS	NOTES
`constitution`	PARTIAL	**Why:** `Constitution.rules` is a general string tuple, but only one rule name (`non_negative_costs`) is ever interpreted — any other string in `rules` is silently ignored. Adoption happens via a direct `adopt_constitution()` call, not the "owner-

		approved event" amendment path §9 requires to avoid self-modification exploits. **Existing support:** model, store, and single-rule evaluator are solid; the extension point is the rule interpreter. **Classification:** extension of Governor.
`policy evaluation`	PARTIAL	**Why:** evaluation is a single `if` branch, not a rule engine — correctly short-circuits before budget checks and emits `PolicyEvaluated`, but can't express more than one invariant. **Existing support:** the gate ordering (policy before budget) and event emission are correct and tested. **Classification:** extension of Governor.
`budget management`	COMPLETE	`BudgetState` reserved atomically on approval, untouched on denial, insufficient-budget path verified independently of the policy gate, `BudgetChecked` carries both required and available figures for audit.

`escalation`	PARTIAL	**Why:** `ApprovalRequiredEvent` fires correctly when no constitution/budget is configured, but nothing consumes it to get a decision back into the system — it's a dead-end signal, not a working owner-in-the-loop path. **Requirement:** "owner escalation" as a real two-way mechanism. **Existing support:** the event and its trigger conditions are correct; this was explicitly scoped out ("owner UI") for the foundation task. **Classification:** new feature.

06 Executor

25%

RESPONSIBILITY	STATUS	NOTES

`retries`	MISSING	**Why:** explicitly out of scope for the foundation task; `ActionAttemptedEvent.attempt` is hardcoded to 1. **Requirement:** Executor owns "retry management" per the architecture table. **Existing support:** the `attempt` field and a clean failure path (`ActionFailed`) are already there to retry against. **Classification:** extension of Executor, intentionally deferred.
`sandboxing`	MISSING	**Why:** tools run as bare in-process Python callables with no isolation boundary. **Requirement:** "sandbox boundaries" is a named Executor responsibility. **Existing support:** the `ToolRegistry` indirection is the natural seam to insert a sandbox at. **Classification:** new feature.
`tool registry`	COMPLETE	Register/get_tool/is_registered; unregistered action types correctly fail closed (recorded + `ActionFailed`) rather than crashing the pipeline.

`credentials`	MISSING	**Why:** no `CredentialStore` or credential-reference mechanism exists anywhere; mock tools take no credentials at all. **Requirement:** Executor owns "credentials" (by reference, per the architecture's ledger/credentials distinction) alongside the tool registry and action log. **Existing support:** none yet. **Classification:** new feature, logically depends on sandboxing existing first.

07 Memory & Ledger 6%

RESPONSIBILITY	STATUS	NOTES

`prediction tracking`	MISSING	**Why:** neither `DecisionRecord` nor `Outcome` carries a predicted value to compare against. **Requirement:** §7, "every decision record stored a predicted outcome." **Existing support:** `DecisionRecord.rationale` is free text where a structured prediction field would go. **Classification:** extension of Executive + Memory & Ledger together.
`outcome comparison`	MISSING	**Why:** depends on prediction tracking, which doesn't exist. **Existing support:** `Outcome.success/result` are the raw material a comparison would consume. **Classification:** extension, blocked on prediction tracking.

`calibration`	MISSING	Explicitly deferred by the Memory & Ledger task. No optimism-discount or per-class calibration logic exists. **Classification:** new feature, blocked on outcome comparison.
`playbooks`	MISSING	Explicitly deferred. `HeuristicStore` exists as scaffolding but nothing ever writes to it. **Classification:** new feature.
`anti-patterns`	MISSING	Explicitly deferred; no mechanism for the Governor to consume distilled anti-patterns as enforceable rules. **Classification:** new feature, pairs with Governor's constitution extension.
`fast learning loop`	PARTIAL	**Why:** World Model's per-observation confidence update exists and works, but without the per-sensor source-reliability tracking §7 specifies — every sensor is trusted equally regardless of track record. **Existing support:**

		`WorldModel._on_observation_created` is the extension point. **Classification:** extension of World Model.
`medium learning loop`	MISSING	**Why:** depends on prediction tracking + outcome comparison, neither built. **Requirement:** per-outcome adjustment of Executive's priors and calibration factor. **Classification:** new feature, blocked.
`slow learning loop`	MISSING	**Why:** depends on the medium loop accumulating scored history first. **Requirement:** pattern-level playbook/anti-pattern distillation. **Classification:** new feature, blocked.

08 Inference Port 100%

RESPONSIBILITY	STATUS	NOTES
provider abstraction	COMPLETE	Abstract `InferenceProvider` with a single `infer()` method; port code never references a provider by name, verified by construction (no OpenAI/Anthropic-shaped fields on `InferencePort`).
provider registry	COMPLETE	`ProviderRegistry` enforces exactly one active provider at a time; swap-at-runtime tested directly.

request routing	COMPLETE	`infer()` routes to the active provider, measures real latency independent of what the provider reports, and emits `InferenceRequested`/`InferenceCompleted` with matching request/response ids.

Note (not a scoring deduction): the port itself is done, but has zero callers — no component in the system invokes `InferencePort.infer()` yet. Executive's expected-value estimation is the natural first caller (see roadmap).

Architecture-wide checks

OWNERSHIP	**Clean.** Every component imports only `src.events` and `src.kernel` — no component imports another component's module. Verified by

	grep across all eight `src/*/__init__.py` files.
CROSS-STORE ACCESS	**Clean.** No component calls another component's store methods directly (e.g. Governor never touches `opportunity_store`). All cross-component data flows through event fields — confirmed by the repeated pattern of extending events (`PlanProposedEvent`, `ApprovalGrantedEvent`, `ActionSucceededEvent/ActionFailedEvent`) rather than reaching into a neighboring store when a gap was found.
EVENT SCHEMA	**Mostly resolved, one debt item.** Three rounds of schema extension made events self-contained end to end (plan costs and ordered_actions flow through Governor to Executor; plan_id flows through Executor to Memory & Ledger). Remaining debt: rationale/reason fields are free-text strings throughout (`DecisionRecord.rationale`, `ApprovalRecord.reason`) rather than structured data, which will make later automated calibration harder to query.

MISSING EVENTS	`PlanAbandonedEvent` and `ExecutiveDecisionEvent` (PLAN_SELECTED) are defined in the taxonomy but never published anywhere — dead scaffolding matching Executive's missing ranking/selection step. No timer-fired event exists, consistent with the Kernel having no timers at all.
CYCLIC DEPS	**None.** The import graph is a clean star: eight components → `events` + `kernel`, nothing else.
PERSISTENCE	**Critical gap.** Zero disk persistence anywhere in the repo — `data/events/`, `data/snapshots/`, `data/stores/` are all empty stub packages. Every store (EventLog included) is pure in-process memory; killing the process loses the entire organism. Directly contradicts axiom 1 ("restart from disk, same organism").
AUDITABILITY	**Structurally good, one wiring gap.** Append-only logs/stores are used consistently and every meaningful state change publishes a correlated event. But `Event.correlation_id` — the field meant to let

	one request chain (observation → belief → opportunity → plan → approval → action → outcome → ledger entry) be traced as a unit — is defined on the base `Event` class and never populated by any publisher. Every event's correlation_id is `None` today.
RECOVERY	Same root cause as persistence — **missing**. No component re-hydrates its store from the event log on startup; there is no startup path that does anything but construct empty stores.
DETERMINISM	**No violations found.** State-transition logic is deterministic given inputs; the only non-determinism is expected (UUIDs, wall-clock timestamps for IDs/audit, not for control flow). Full replay-determinism is unverified since replay doesn't exist yet.
COMPOSITION ROOT	**Missing.** `main.py` is a no-op stub. No code anywhere instantiates all eight components against one shared `Kernel` and runs them together — every component has only ever been exercised in isolation inside its

own unit tests. The full Observe → Deliberate → Governor gate → Execute → Measure loop has never actually run end to end.

Ranked roadmap

Ordered so each item only depends on items above it.

1 Populate `correlation_id` on every publish

Cheap, no dependencies, touches every publish call site once. Turns the audit trail from "traceable in principle" into actually traceable. Unblocks: every later auditability/calibration item.

---

2 Kernel: disk persistence for the event log

Serialize `EventLog` entries as they're appended. Foundational — nothing durable can exist above it. Unblocks: replay, crash recovery, snapshotting.

3

Kernel: replay-on-start

Each component re-subscribes and re-derives its store from the persisted log tail instead of starting empty. Depends on: persistence (above).

4

Kernel: snapshotting

Periodic state snapshot so replay doesn't mean reading the log from event zero forever. Depends on: persistence + replay.

5

Kernel: scheduler + timer table

A real "fire at/after T" primitive. Unblocks: automatic belief decay (currently manually invoked), and any periodic learning-loop work later.

6 Wire the composition root

Build `main.py` into an actual assembly: one Kernel, all eight components constructed against it, `kernel.start()` called, one real Observe→Measure cycle proven to run outside a test. Depends on: nothing new, but this is the first point the whole organism is provably alive.

7 Perception: real normalization adapters

Replace the normalized-value pass-through with actual raw→normalized transforms per sensor type. Depends on: composition root existing, so there's a real signal source to normalize.

8 Executive ↔ Governor: capital/attention allocation against live budget

Executive reads Governor's `BudgetChecked/ApprovalGranted` history before proposing, instead of guessing fixed costs. Depends on: composition root (needs a live loop to allocate within).

---

9 Executive: Inference Port integration for EV estimation

Replace `confidence × 100` with a real judgment call through the now-complete Inference Port (mock provider is enough to start). Depends on: nothing technically new — Inference Port is done — but pairs naturally with the allocation work above.

---

10 Executive: ranking + plan selection

Generate multiple candidate plans, rank by EV/cost, wire up the already-defined but unused `PlanAbandonedEvent` / `ExecutiveDecisionEvent`. Depends on: real EV estimation (above) — ranking is meaningless against a constant formula.

---

11 Governor: real rule engine + owner-approved amendment path

Move past the single hardcoded `non_negative_costs` check; gate constitution changes behind an owner-approved event instead of a raw method call. Depends on: Executive producing more varied plans worth constraining (item 10).

---

12 Governor: real escalation path

Something has to consume `ApprovalRequired` and feed an owner decision back in — today it's a dead end. Depends on: nothing new, more valuable once real judgment calls exist (item 10).

---

13 Executor: sandboxing boundary

Isolate tool execution before trusting it with retries or credentials. Depends on: nothing new, sequenced before the next two items deliberately.

---

14 Executor: retries

Bounded retry/backoff on `ActionFailed`, using the existing `attempt` field. Depends on: sandboxing — retrying inside an unsandboxed tool multiplies the risk.

---

15 Executor: credentials

`CredentialStore` with reference-only access for tools. Depends on: sandboxing — credentials shouldn't reach unsandboxed code.

---

16 Memory & Ledger + Executive: prediction tracking

`DecisionRecord` gains a structured predicted-value field; `Outcome`/`OutcomeRecorded` carries what to compare it against. Depends on: real EV estimation (item 9) — comparing against a constant formula isn't meaningful.

---

17 Memory & Ledger: medium learning loop (calibration)

Score prediction vs. reality, adjust Executive's priors and optimism discount. Depends on: prediction tracking (above) and the scheduler (item 5) if run periodically rather than per-outcome.

---

18 **Memory & Ledger: slow learning loop (playbooks / anti-patterns)**

Distill repeated successes/failures into reusable procedures and Governor-enforceable anti-patterns. Depends on: medium loop accumulating enough scored history first.

Items intentionally deferred

Pulled directly from each foundation task's own "Do NOT implement" list — consciously postponed because they depended on infrastructure that didn't exist yet when that task was scoped, not oversights.

FROM THE PERCEPTION TASK

- Beliefs, confidence updating, provenance reasoning, decay — picked up by World Model (now complete).
- World Model, Executive, Governor themselves — all built in later tasks.
- Learning — still pending, see Memory & Ledger row above.

FROM THE WORLD MODEL TASK

- Opportunities, planning, budgets, tool execution — picked up by Executive, Governor, and Executor.
- Learning loops, LLM inference — still pending; Memory & Ledger and Inference Port exist but aren't wired to World Model yet.

FROM THE EXECUTIVE TASK

- Approval, budgets, constitution — picked up by Governor (now complete for its own scope).
- Execution, tools — picked up by Executor.
- Learning, LLM inference — still pending; Executive doesn't call the Inference Port yet.

FROM THE GOVERNOR TASK

- Execution, tool calls — picked up by Executor.
- Memory — partially picked up by Memory & Ledger.
- Retries, learning, inference, owner UI — still pending.

FROM THE EXECUTOR TASK

- Memory — picked up by Memory & Ledger.
- External APIs — permanently out of scope by design (mock tools only), not "pending."
- Retry scheduling, sandboxing, credentials, learning, inference — still pending.

FROM THE MEMORY & LEDGER TASK

- Playbooks, anti-patterns, calibration, prediction scoring, Executive updates, learning loops, inference — all still pending; this is the largest deferred backlog in the system.

FROM THE INFERENCE PORT TASK

- Real provider APIs (OpenAI, Anthropic, Gemini, Ollama, vLLM) — permanently out of scope for this task by design, not "pending" the way learning loops are.
- Prompt engineering, learning, caching, retries — still pending as infrastructure concerns once a real provider is added.

spawn/ — read-only architecture audit — no code modified — 134/134 tests passing at time of audit